

3.2 神经网络语言模型

(RNN/LSTM/GRU)

陈 露

上海交通大学计算机科学与工程系
2023年秋季学期

- ▶ 语言模型基本概念
- ▶ 统计语言模型回顾
- ▶ 神经网络语言模型
 - ▶ 前馈神经网络
 - ▶ 循环神经网络RNN
- ▶ 长短时记忆网络LSTM及GRU
- ▶ 不同语言模型效果对比
- ▶ RNN/LSTM/GRU的应用

- ▶ 语言模型基本概念
- ▶ 统计语言模型回顾
- ▶ 神经网络语言模型
 - ▶ 前馈神经网络
 - ▶ 循环神经网络
- ▶ 长短时记忆网络LSTM及GRU
- ▶ 不同语言模型效果对比
- ▶ RNN/LSTM/GRU的应用

日常生活中的语言模型



基本任务：已知前文预测下一个字

- ▶ 输入：单词序列 $w_1, w_2, \dots, w_{k-1}$
- ▶ 输出：对下一个单词预测的概率 $P(w_k | w_1, \dots, w_{k-1})$

基本任务：已知前文预测下一个字

- ▶ 输入：单词序列 $w_1, w_2, \dots, w_{k-1}$
- ▶ 输出：对下一个单词预测的概率 $P(w_k | w_1, \dots, w_{k-1})$

另一个角度：推测句子的合理程度

- ▶ $\prod_{k=1}^K P(w_k | w_1, \dots, w_{k-1}) = P(w_1, w_2, \dots, w_K)$

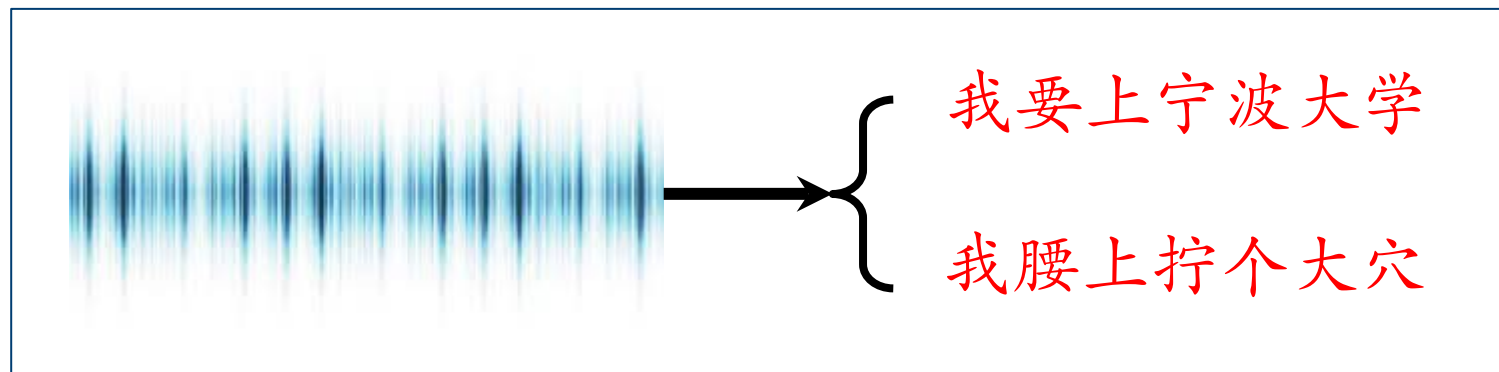
语言模型基本概念

基本任务：已知前文预测下一个字

- ▶ 输入：单词序列 $w_1, w_2, \dots, w_{k-1}$
- ▶ 输出：对下一个单词预测的概率 $P(w_k | w_1, \dots, w_{k-1})$

另一个角度：推测句子的合理程度

- ▶ $\prod_{k=1}^K P(w_k | w_1, \dots, w_{k-1}) = P(w_1, w_2, \dots, w_K)$



- ▶ 语言模型基本概念
- ▶ 统计语言模型回顾
- ▶ 神经网络语言模型
 - ▶ 前馈神经网络
 - ▶ 循环神经网络
- ▶ 长短时记忆网络LSTM及GRU
- ▶ 不同语言模型效果对比
- ▶ RNN/LSTM/GRU的应用

统计语言模型回顾：n-gram

- ▶ 马尔科夫假设：每个字出现的概率仅与其前面出现的 $n-1$ 个字有关。
- ▶ 基本思想：用出现频率代替出现概率
- ▶ 公式：

$$\begin{aligned} P(w_k | w_1, \dots, w_{k-1}) &\approx P(w_k | w_{k-n+1}, \dots, w_{k-1}) \\ &= \frac{P(w_{k-n+1}, \dots, w_{k-1}, w_k)}{P(w_{k-n+1}, \dots, w_{k-1})} \end{aligned}$$

- ▶ 实现：
 - ▶ 对数据集中的所有n-gram, (n-1)-gram计数并存储
- $$\frac{P(w_{k-n+1}, \dots, w_{k-1}, w_k)}{P(w_{k-n+1}, \dots, w_{k-1})} = \frac{\text{Count}(w_{k-n+1}, \dots, w_{k-1}, w_k)}{\text{Count}(w_{k-n+1}, \dots, w_{k-1})}$$

- ▶ Storage Problem
 - ▶ 模型需要存储的数据规模为 $|V|^n$ 量级
 - ▶ 不能建模远距依赖关系

n-gram模型的不足

► Storage Problem

- 模型需要存储的数据规模为 $|V|^n$ 量级
- 不能建模远距依赖关系

He grew up in France, so he speaks French fluently.

二元n-gram语言模型：

$p(\text{French} \mid \text{He grew up in France, so he speaks}) = p(\text{French} \mid \text{speaks})$

$p(\text{French} \mid \text{He grew up in China, so he speaks}) = p(\text{French} \mid \text{speaks})$

n-gram模型的不足

- ▶ Storage Problem
 - ▶ 模型需要存储的数据规模为 $|V|^n$ 量级
 - ▶ 不能建模远距依赖关系
- ▶ Sparsity Problem
 - ▶ 低频次
 - ▶ 零概率
 - ▶ Discounting & Backing-off

n-gram模型的不足

- ▶ Storage Problem
 - ▶ 模型需要存储的数据规模为 $|V|^n$ 量级
 - ▶ 不能建模远距依赖关系
- ▶ Sparsity Problem
 - ▶ 低频次
 - ▶ 零概率
 - ▶ Discounting & Backing-off

- ▶ 语言模型基本概念
- ▶ 统计语言模型回顾
- ▶ 神经网络语言模型
 - ▶ 前馈神经网络
 - ▶ 循环神经网络
- ▶ 长短时记忆网络LSTM及GRU
- ▶ 不同语言模型效果对比
- ▶ RNN/LSTM/GRU的应用

- ▶ 直接参考n-gram模型的方法，将历史词的词向量拼接起来送入前馈神经网络并计算下一个词的概率

前馈神经网络语言模型的计算流程

我 超 级 喜 欢 这 部

前馈神经网络语言模型的计算流程

喜 欢 这 部

前馈神经网络语言模型的计算流程

各个字的单热向量
 $x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$

喜
 $x^{(1)}$

欢
 $x^{(2)}$

这
 $x^{(3)}$

部
 $x^{(4)}$

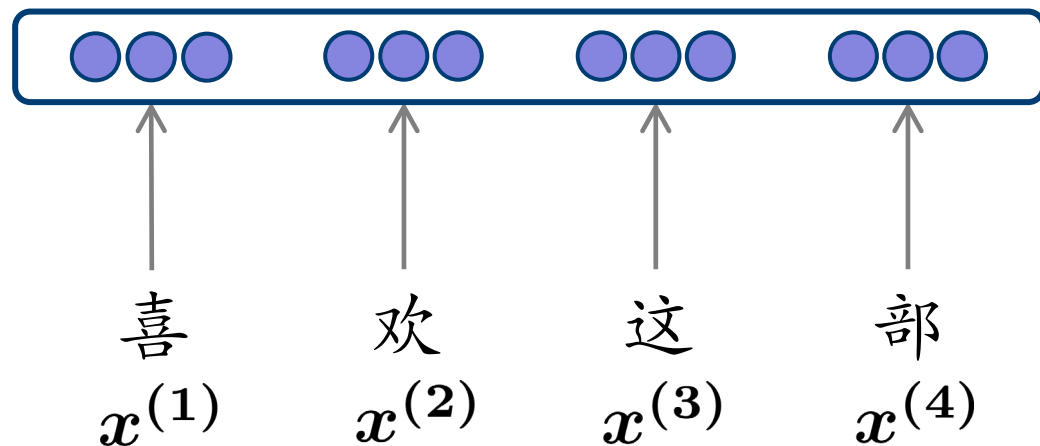
前馈神经网络语言模型的计算流程

查询拼接相应词向量

$$e = [e^{(1)}, e^{(2)}, e^{(3)}, e^{(4)}]$$

各个字的单热向量

$$x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$$



前馈神经网络语言模型的计算流程

隐藏层

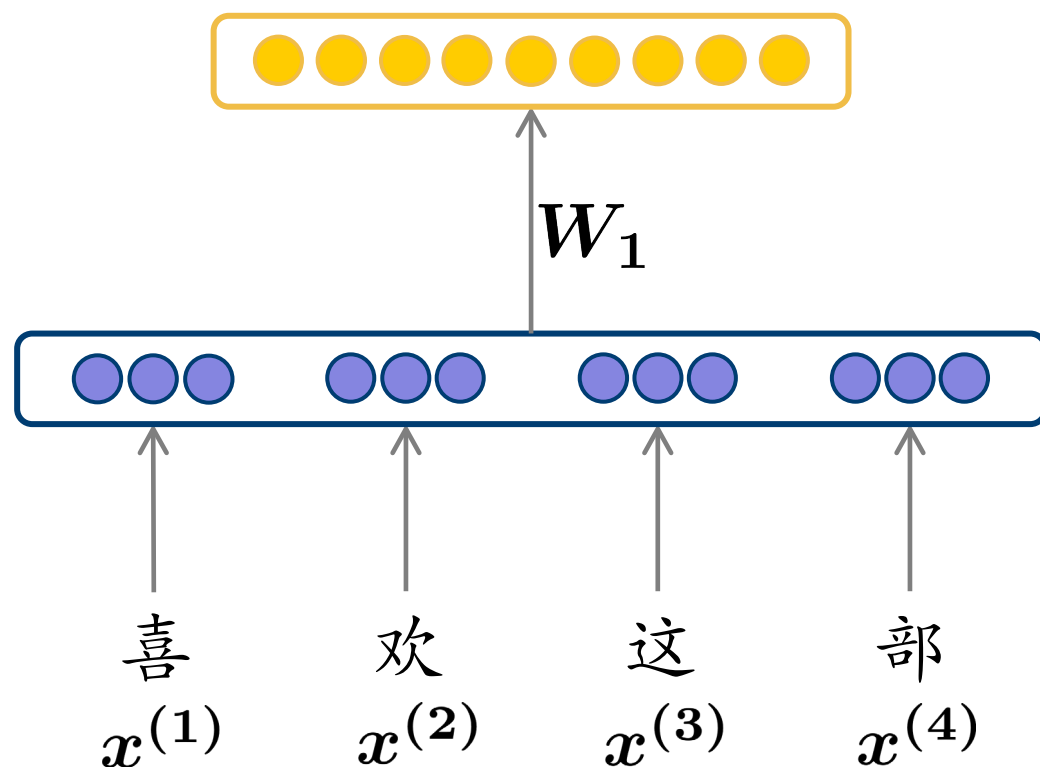
$$h = f(W_1 e + b_1)$$

查询拼接相应词向量

$$e = [e^{(1)}, e^{(2)}, e^{(3)}, e^{(4)}]$$

各个字的单热向量

$$x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$$



前馈神经网络语言模型的计算流程

输出概率分布

$$\hat{y} = \text{Softmax}(W_2 h + b_2)$$

隐藏层

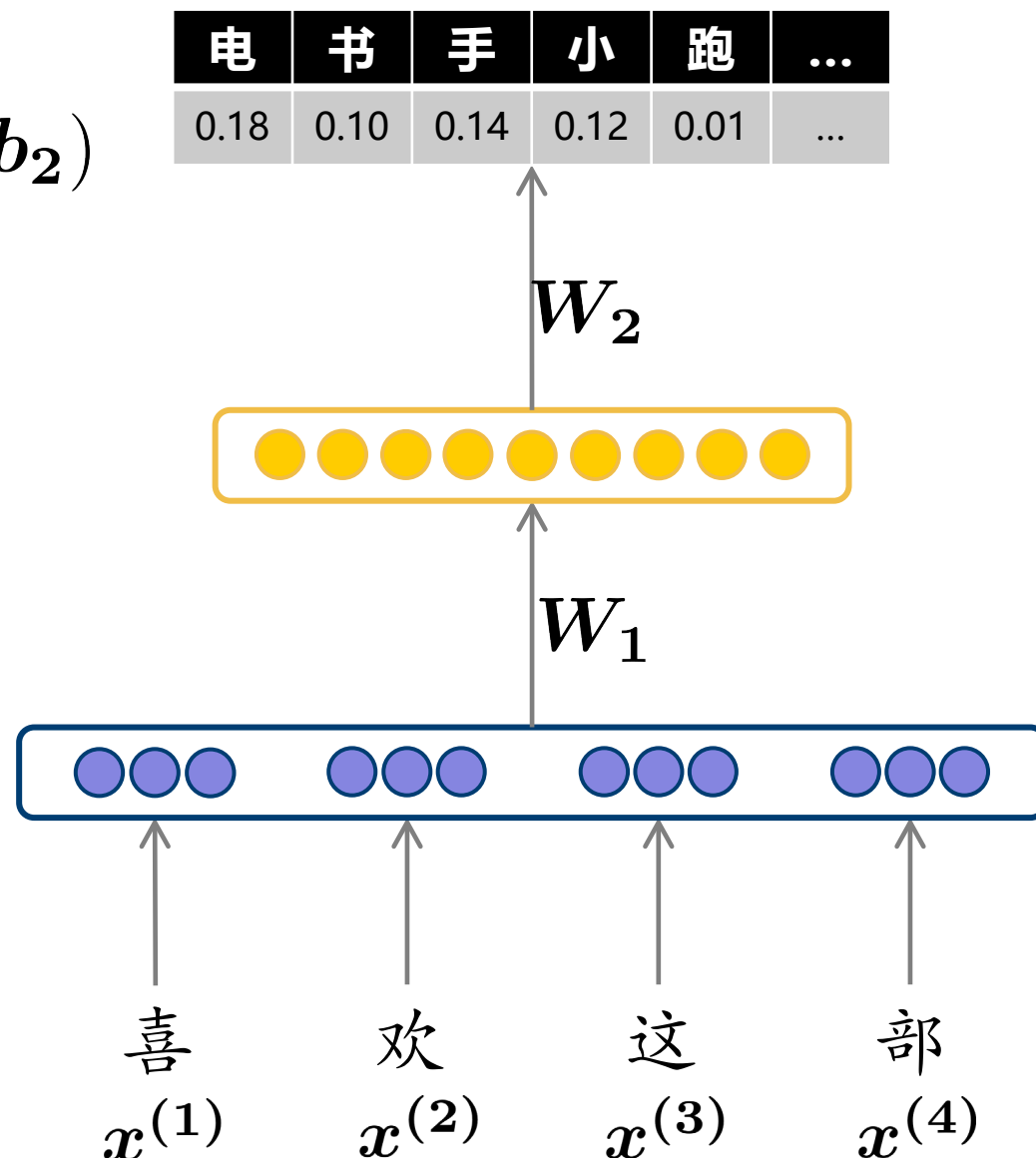
$$h = f(W_1 e + b_1)$$

查询拼接相应词向量

$$e = [e^{(1)}, e^{(2)}, e^{(3)}, e^{(4)}]$$

各个字的单热向量

$$x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$$



前馈神经网络语言模型的优点

- ▶ 通过学习单词分布式表示，有效缓解数据稀疏问题
- ▶ 避免维度灾难，相比n-gram模型占用更小的存储空间
- ▶ 可以并行计算不同时刻单词

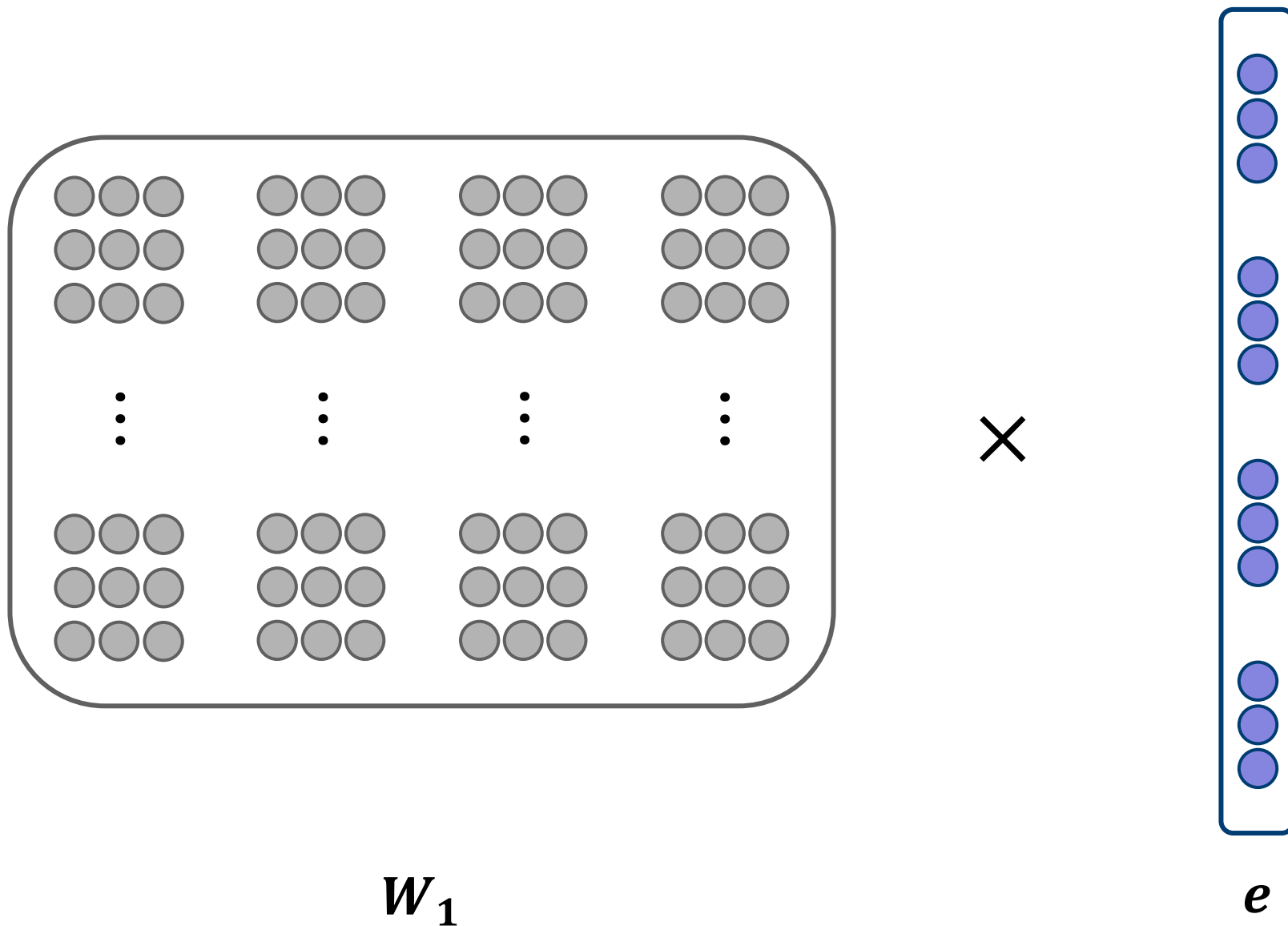
前馈神经网络语言模型的问题

- ▶ 前馈神经网络需要保存的参数
 - ▶ 词向量矩阵 $E \in \mathbb{R}^{|\mathcal{V}| \times d}$
 - ▶ 权重矩阵 $W_1 \in \mathbb{R}^{d_h \times (n \times d)}$
 - ▶ 输出矩阵 $W_2 \in \mathbb{R}^{|\mathcal{V}| \times d_h}$

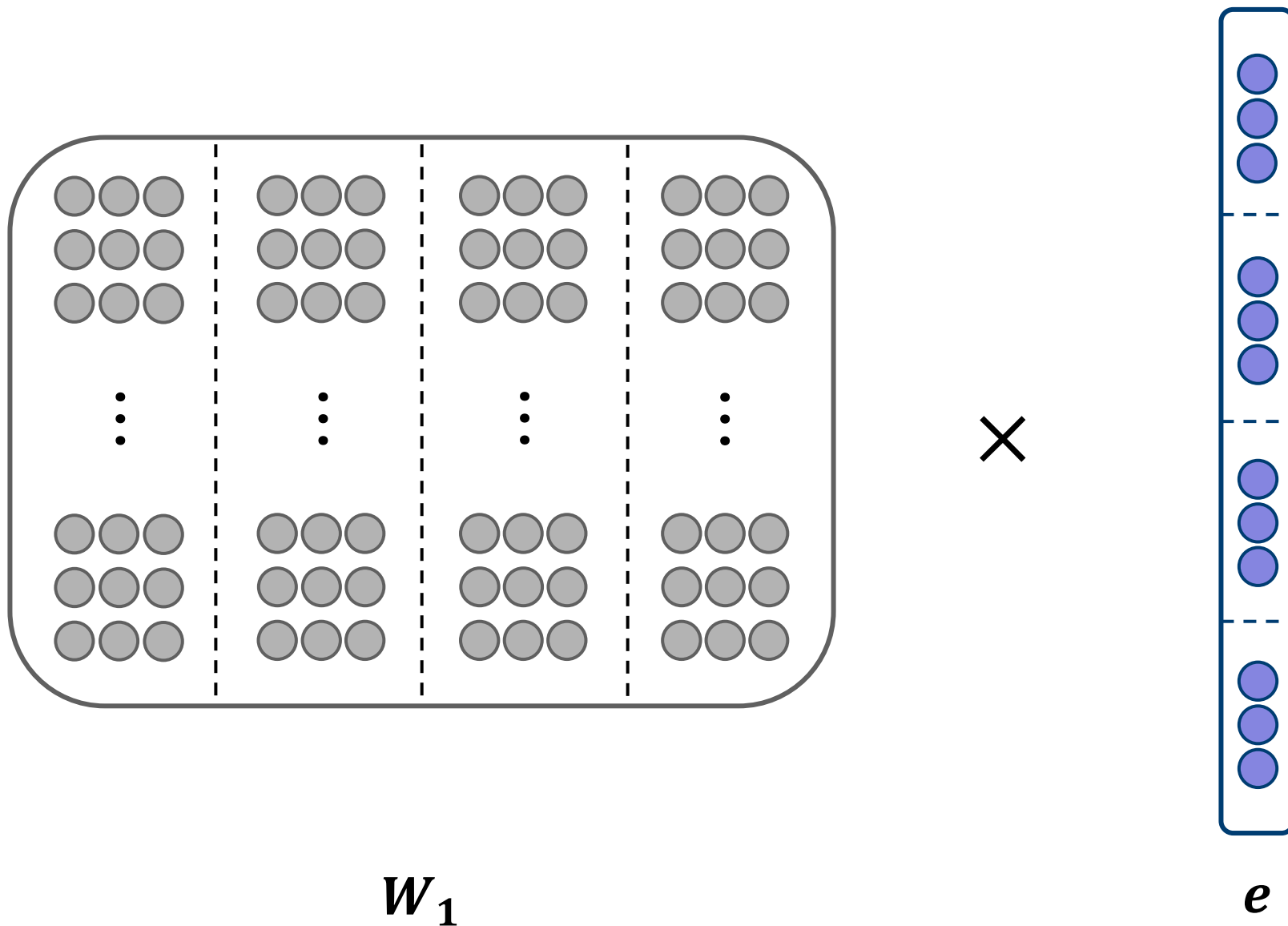
前馈神经网络语言模型的问题

- ▶ 前馈神经网络需要保存的参数
 - ▶ 词向量矩阵 $E \in \mathbb{R}^{|\mathcal{V}| \times d}$
 - ▶ 权重矩阵 $W_1 \in \mathbb{R}^{d_h \times (n \times d)}$
 - ▶ 输出矩阵 $W_2 \in \mathbb{R}^{|\mathcal{V}| \times d_h}$
- ▶ 权重矩阵的大小仍会随着 n 的增大而增大
 - ▶ 不能完全解决Storage Problem
 - ▶ 仍不能很好的建模长时依赖

观察 $W_1 \times e$ 的计算



观察 $W_1 \times e$ 的计算



观察 $W_1 \times e$ 的计算

$$\begin{bmatrix} W_1^{(1)}; & W_1^{(2)}; & W_1^{(3)}; & W_1^{(4)} \end{bmatrix} \times \begin{bmatrix} e^{(1)} \\ e^{(2)} \\ e^{(3)} \\ e^{(4)} \end{bmatrix} = \begin{aligned} &W_1^{(1)}e^{(1)} \\ &+ \\ &W_1^{(2)}e^{(2)} \\ &+ \\ &W_1^{(3)}e^{(3)} \\ &+ \\ &W_1^{(4)}e^{(4)} \end{aligned}$$

重构权重矩阵——循环神经网络的引出

- ▶ 权重矩阵的问题
 - ▶ 需要单独对每一个历史时刻的词学习参数
 - ▶ 不同时刻间共通的处理要不断重复学习

重构权重矩阵——循环神经网络的引出

- ▶ 权重矩阵的问题
 - ▶ 需要单独对每一个历史时刻的词学习参数
 - ▶ 不同时刻间共通的处理要不断重复学习
- ▶ 分解权重矩阵 $\mathbf{W}_1 \in \mathbb{R}^{d_h \times (n \times d)}$
 - ▶ 不同时刻间相同的处理
 - ▶ $\mathbf{W}_e \in \mathbb{R}^{d_h \times d}$
 - ▶ 不同时刻间不同的处理
 - ▶ 假定已有信息向下一时刻传递时的处理是相同的
 - ▶ $\mathbf{W}_h \in \mathbb{R}^{d_h \times d_h}$

重构权重矩阵——循环神经网络的引出

- ▶ 权重矩阵的问题
 - ▶ 需要单独对每一个历史时刻的词学习参数
 - ▶ 不同时刻间共通的处理要不断重复学习
- ▶ 分解权重矩阵 $\mathbf{W} \Rightarrow \mathbf{W}_e \in \mathbb{R}^{d_h \times d}, \mathbf{W}_h \in \mathbb{R}^{d_h \times d_h}$
 - ▶ 不再与历史长度 n 相关
- ▶ 从另一个角度理解
 - ▶ 在时间维度上大量使用共享参数的思想

循环神经网络语言模型的计算流程

各个字的单热向量 $x^{(t)}$

喜
 $x^{(1)}$

欢
 $x^{(2)}$

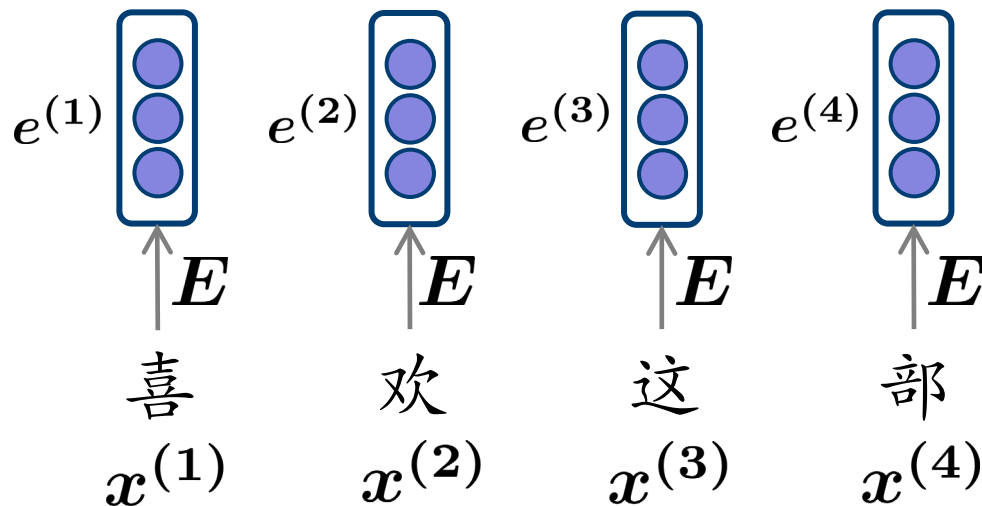
这
 $x^{(3)}$

部
 $x^{(4)}$

循环神经网络语言模型的计算流程

t 时刻词向量
 $e^{(t)} = Ex^{(t)}$

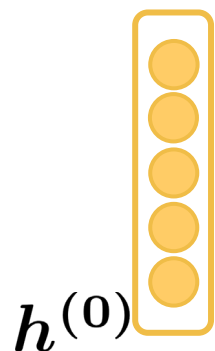
各个字的单热向量 $x^{(t)}$



循环神经网络语言模型的计算流程

隐藏状态

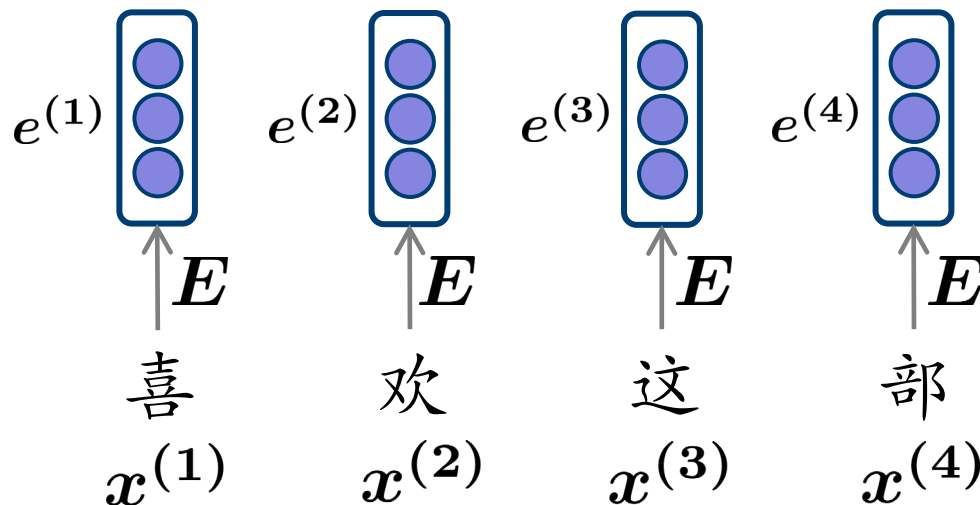
$h^{(0)}$ 是初始隐藏状态



t 时刻词向量

$$e^{(t)} = Ex^{(t)}$$

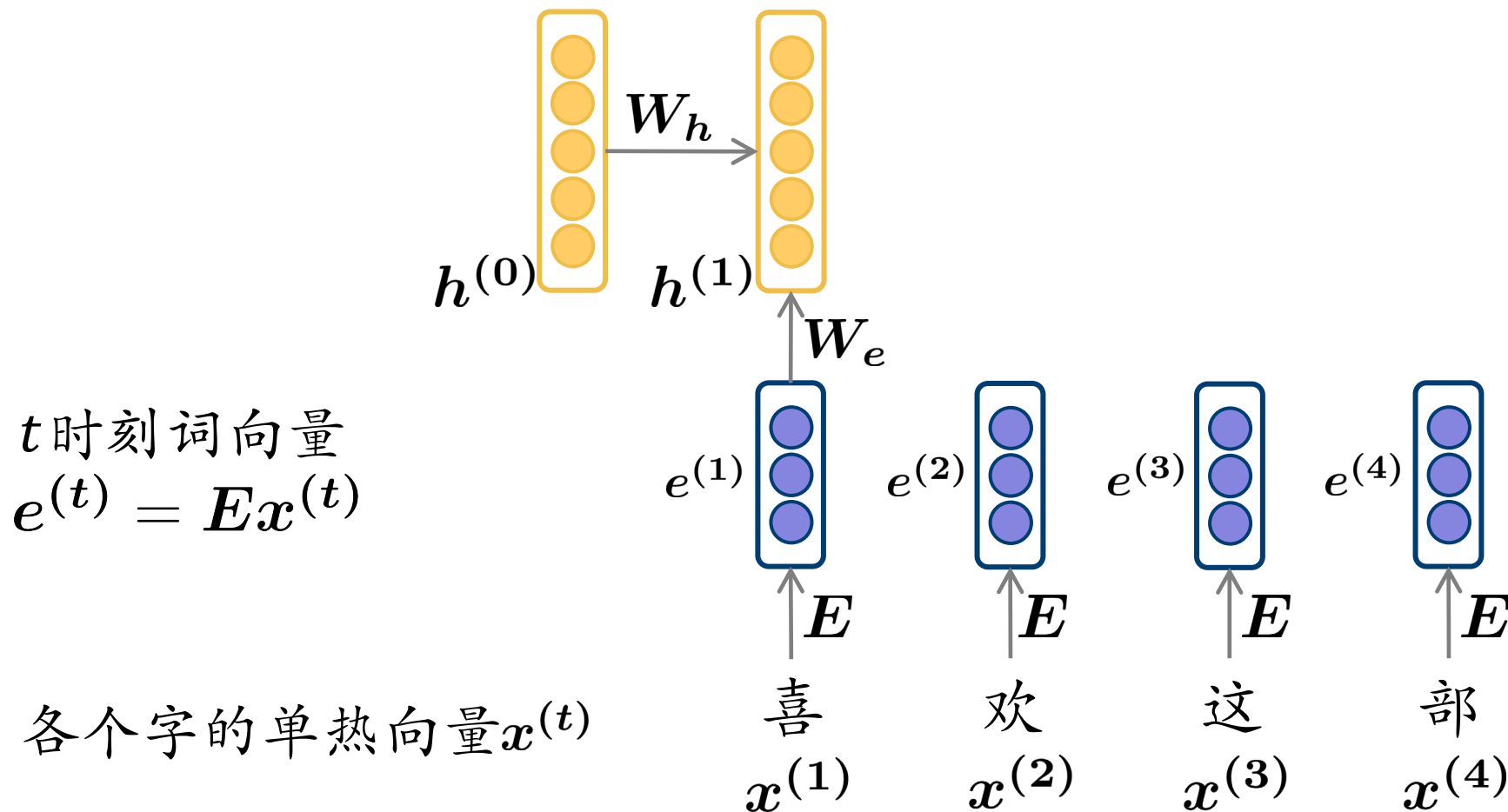
各个字的单热向量 $x^{(t)}$



循环神经网络语言模型的计算流程

隐藏状态

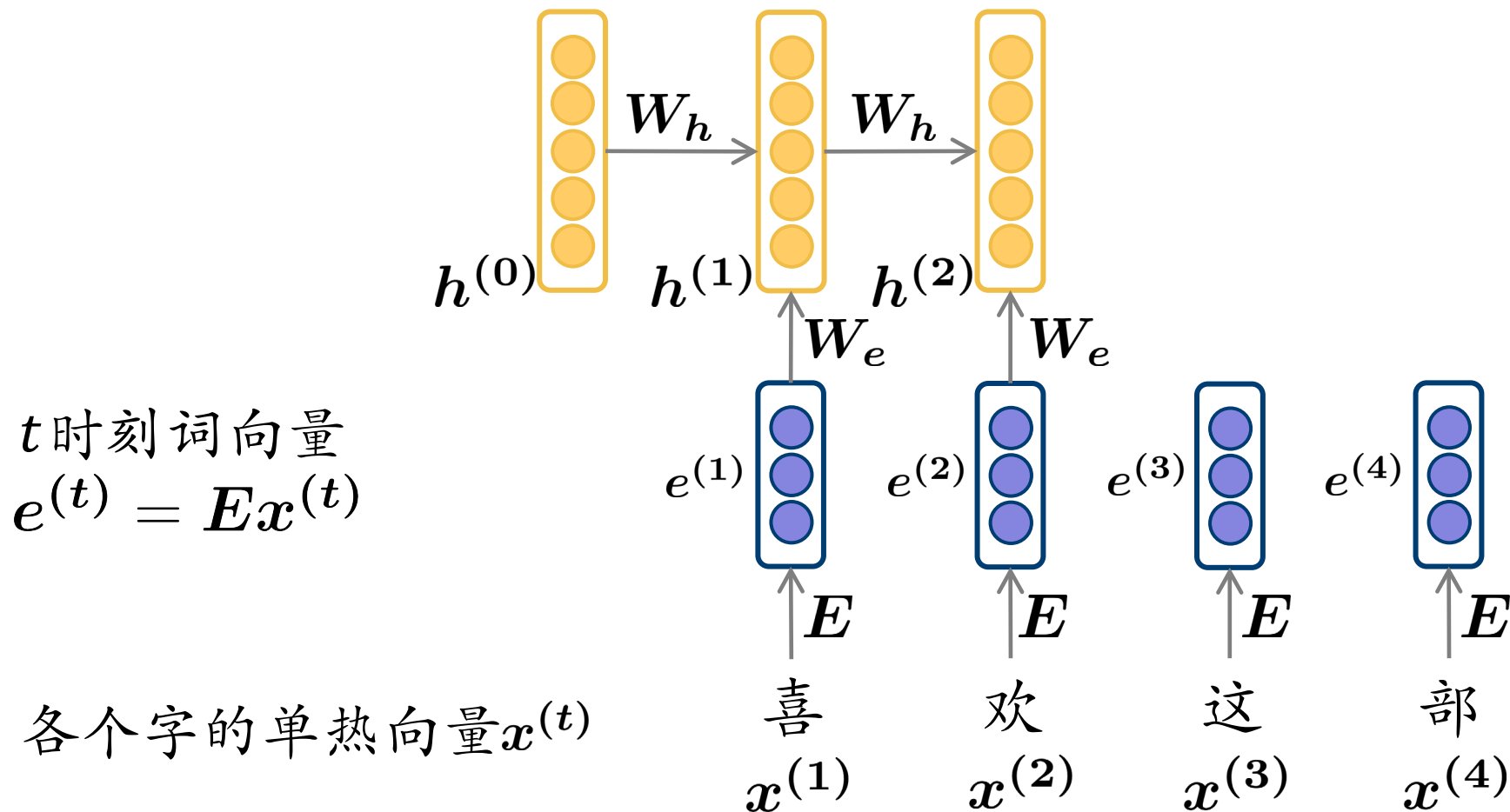
$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$



循环神经网络语言模型的计算流程

隐藏状态

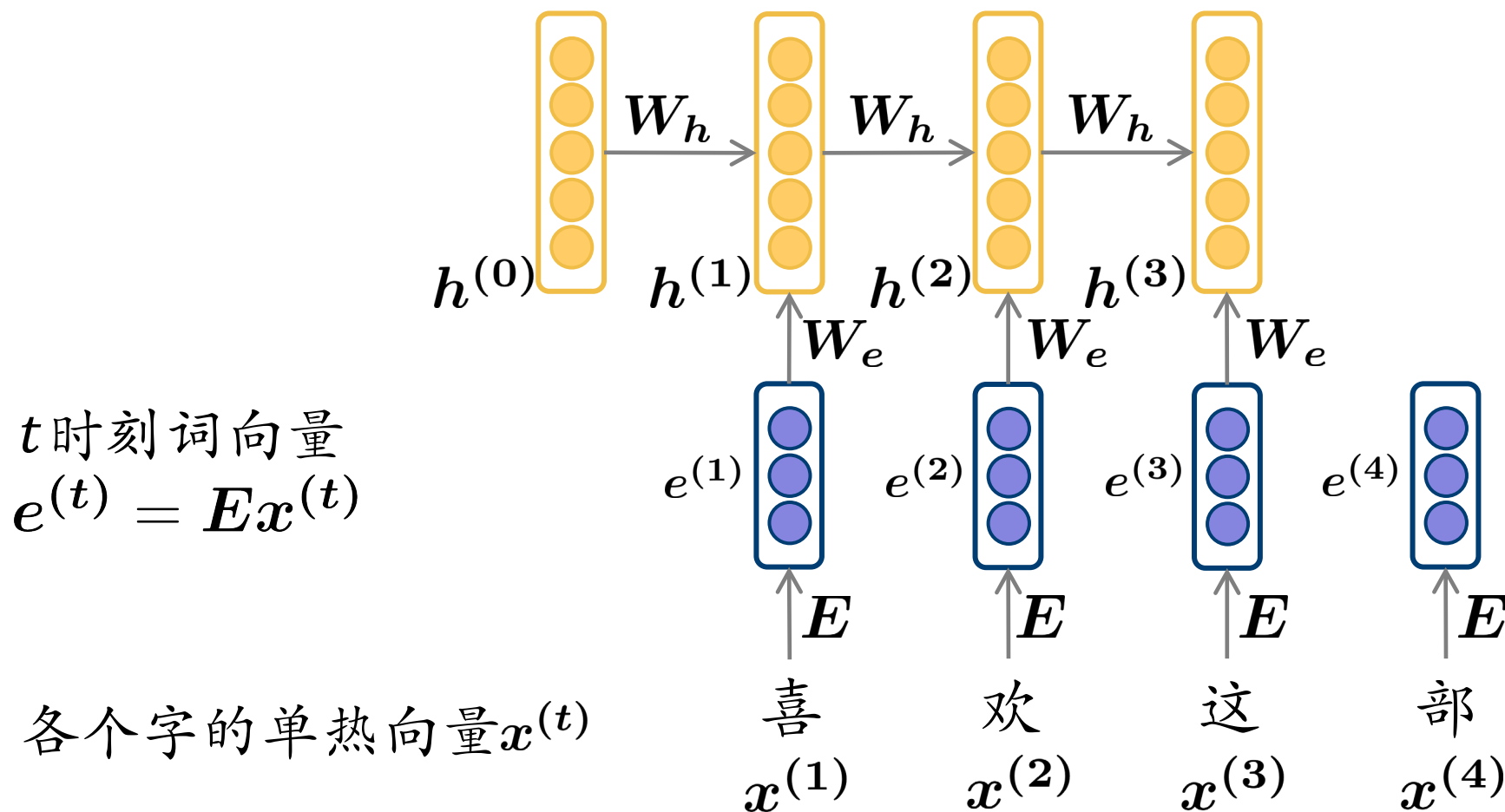
$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$



循环神经网络语言模型的计算流程

隐藏状态

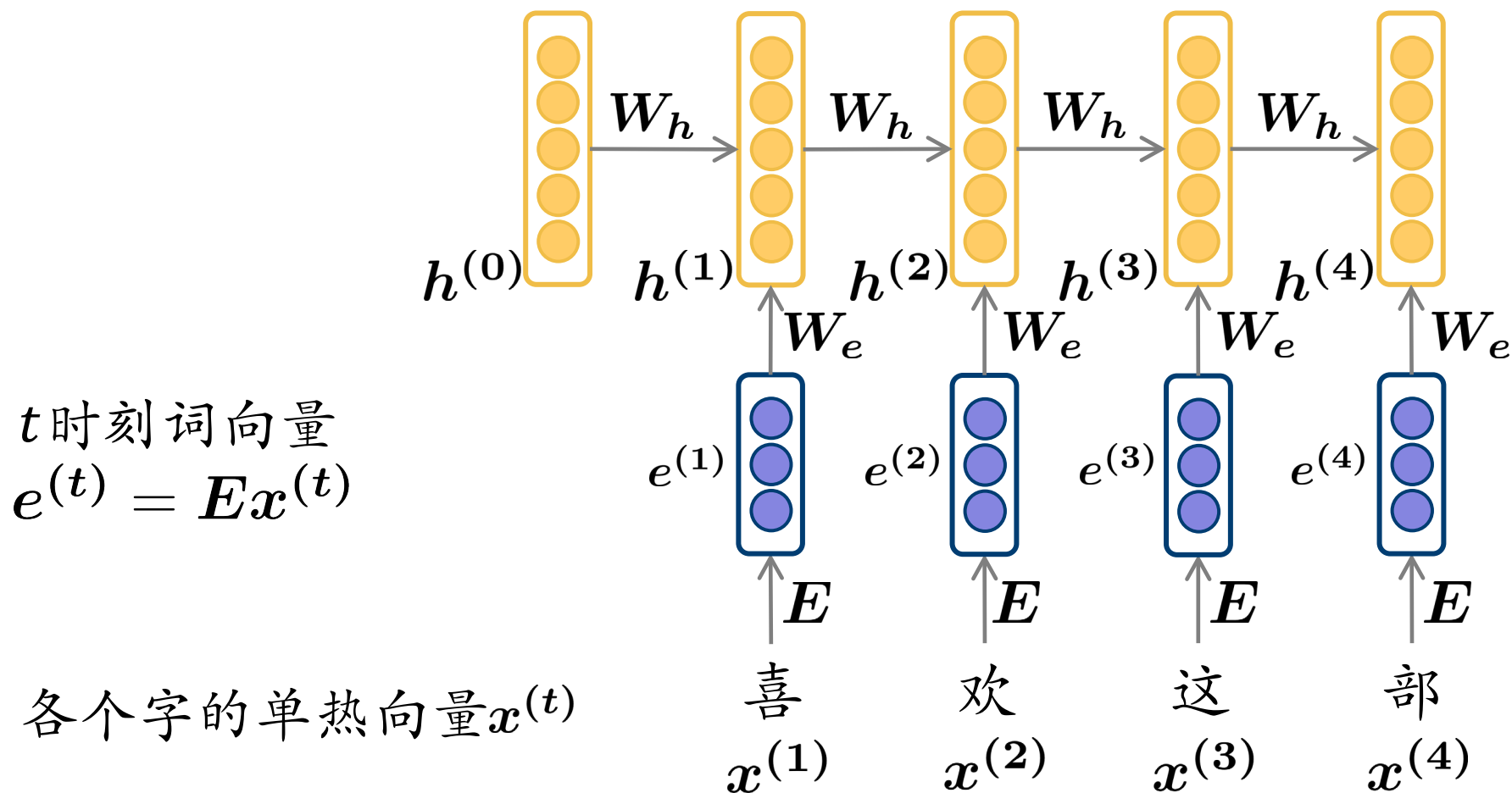
$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$



循环神经网络语言模型的计算流程

隐藏状态

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$



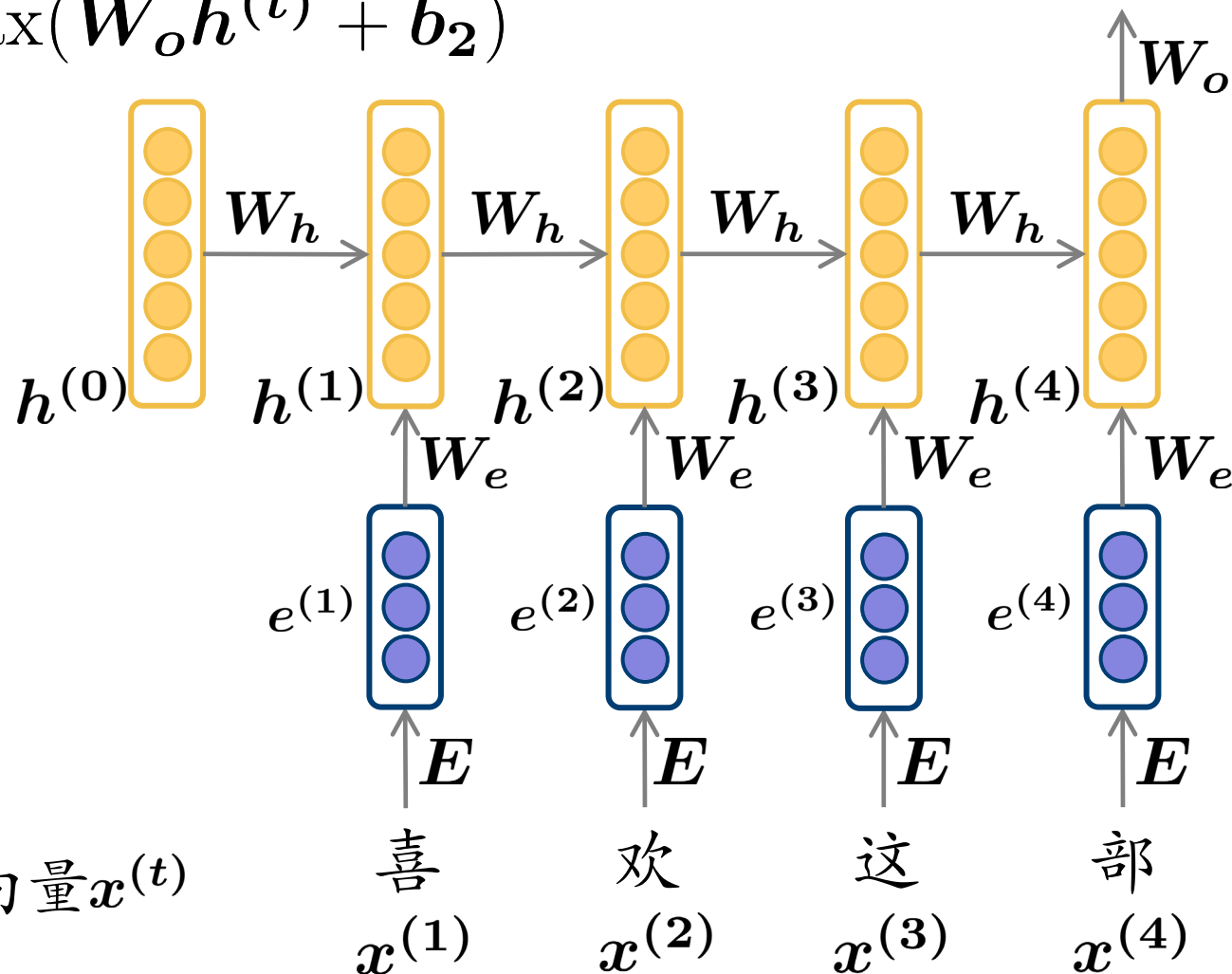
循环神经网络语言模型的计算流程

输出概率分布

$$\hat{y}^{(t)} = \text{Softmax}(W_o h^{(t)} + b_2)$$

电	书	手	小	跑	...
0.18	0.10	0.14	0.12	0.01	...

隐藏状态 $h^{(t)}$



t 时刻词向量
 $e^{(t)} = Ex^{(t)}$

各个字的单热向量 $x^{(t)}$

- ▶ 采用交叉熵损失函数

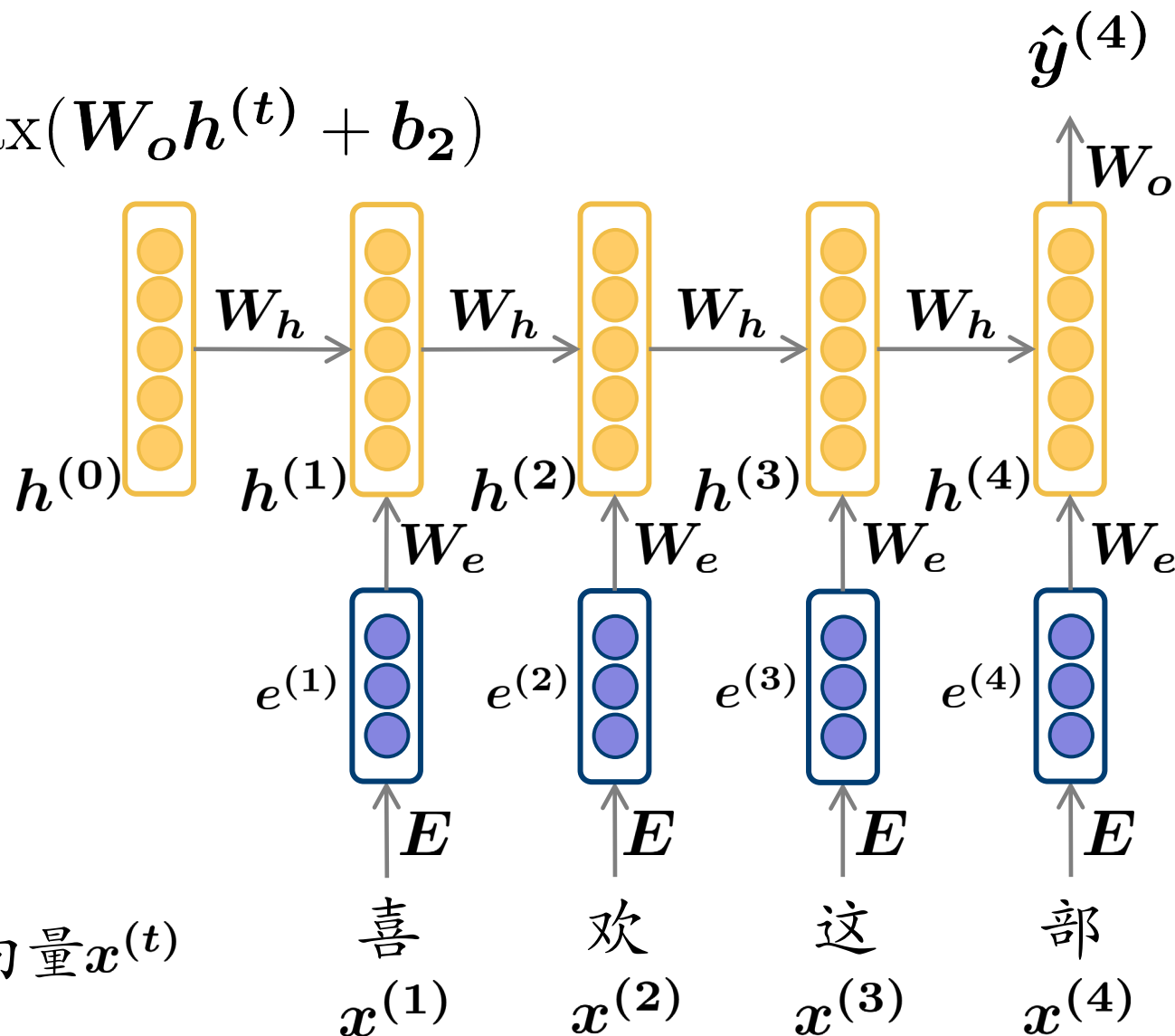
$$\mathcal{L}^{(t)}(\theta) = - \sum_{w \in \mathcal{V}} p_w^{(t)} \log \hat{p}_w^{(t)} = - \log \hat{p}_{w_{t+1}}^{(t)}$$

循环神经网络语言模型的计算流程

输出概率分布

$$\hat{y}^{(t)} = \text{Softmax}(W_o h^{(t)} + b_2)$$

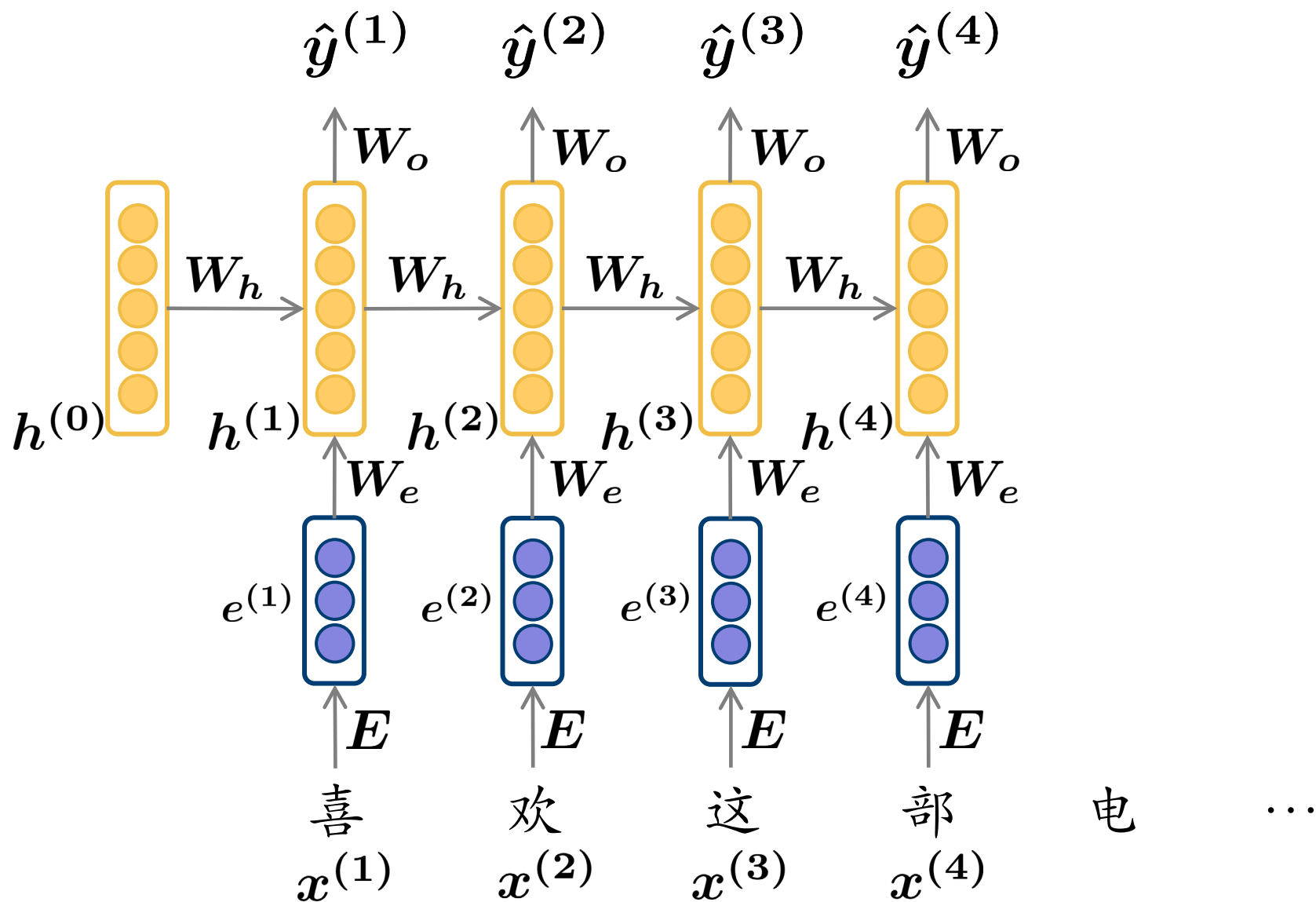
隐藏状态 $h^{(t)}$



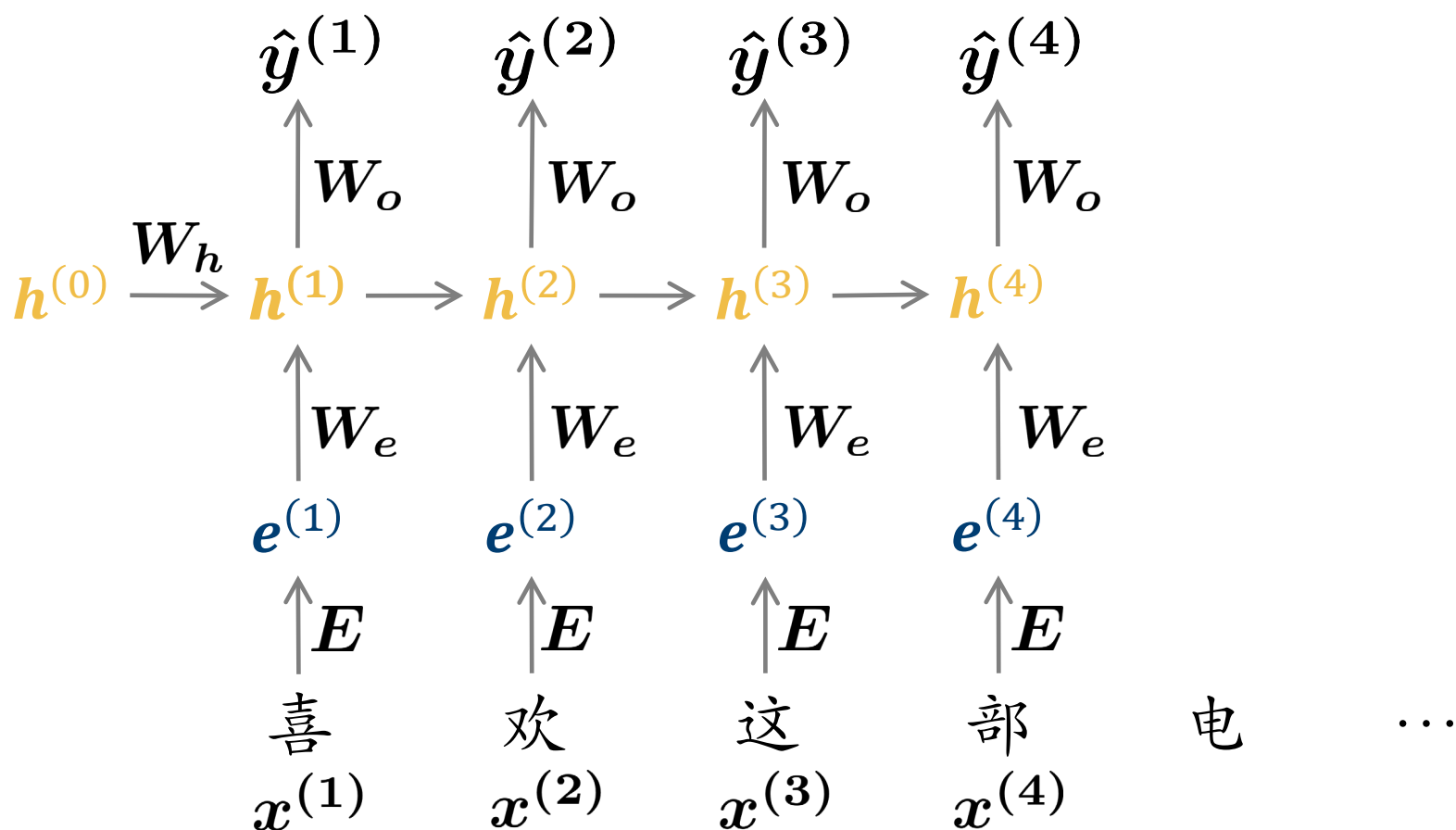
t 时刻词向量
 $e^{(t)} = Ex^{(t)}$

各个字的单热向量 $x^{(t)}$

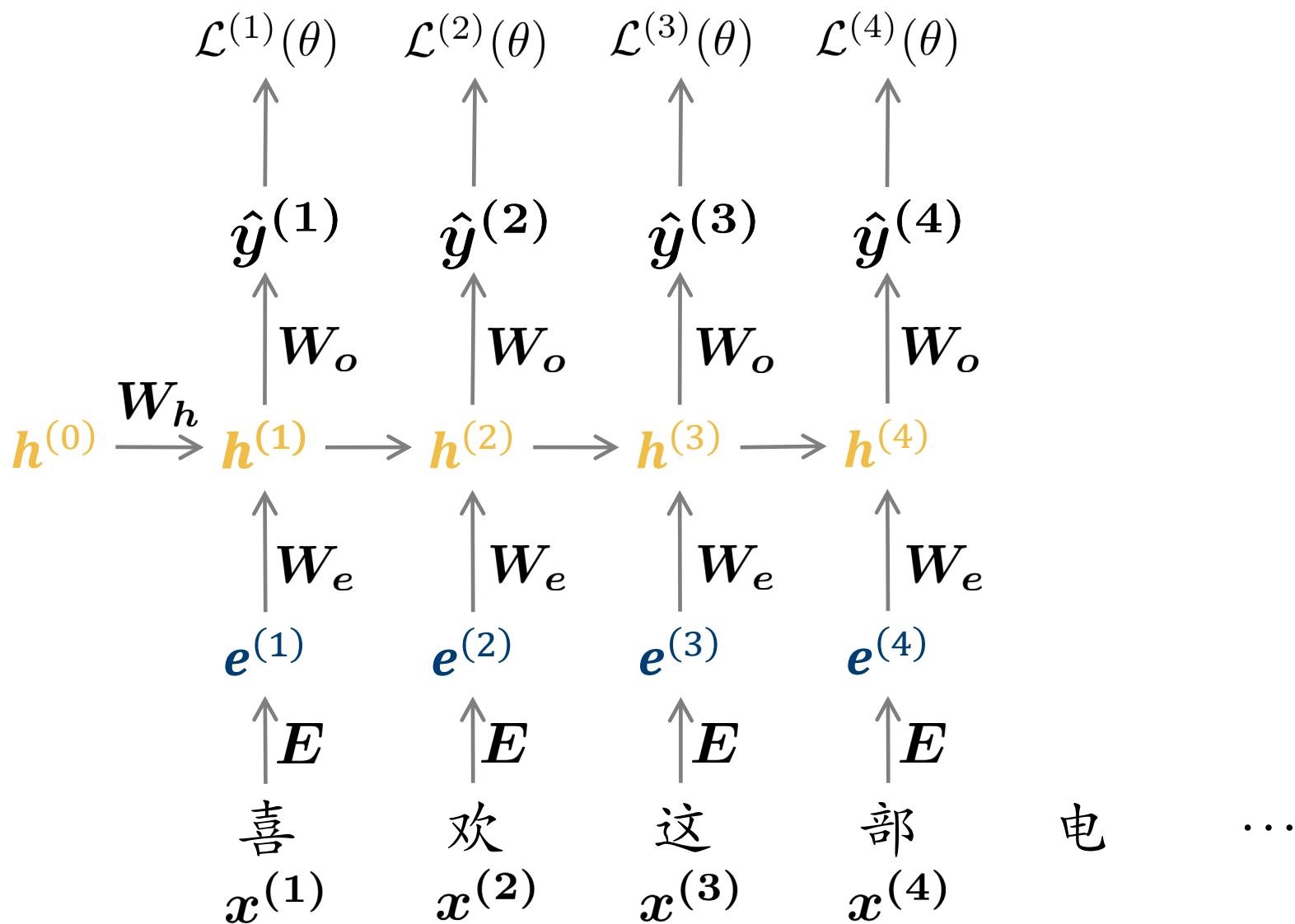
循环神经网络的训练



循环神经网络的训练



循环神经网络的训练



- ▶ 采用交叉熵损失函数

$$\mathcal{L}^{(t)}(\theta) = - \sum_{w \in \mathcal{V}} p_w^{(t)} \log \hat{p}_w^{(t)} = - \log \hat{p}_{w_{t+1}}^{(t)}$$

- ▶ 总的损失函数为：

$$\mathcal{L}(\theta) = \frac{1}{T} \sum_{t=1}^T \mathcal{L}^{(t)}(\theta) = - \frac{1}{T} \sum_{t=1}^T \log \hat{p}_{w_{t+1}}^{(t)}$$

循环神经网络的梯度计算

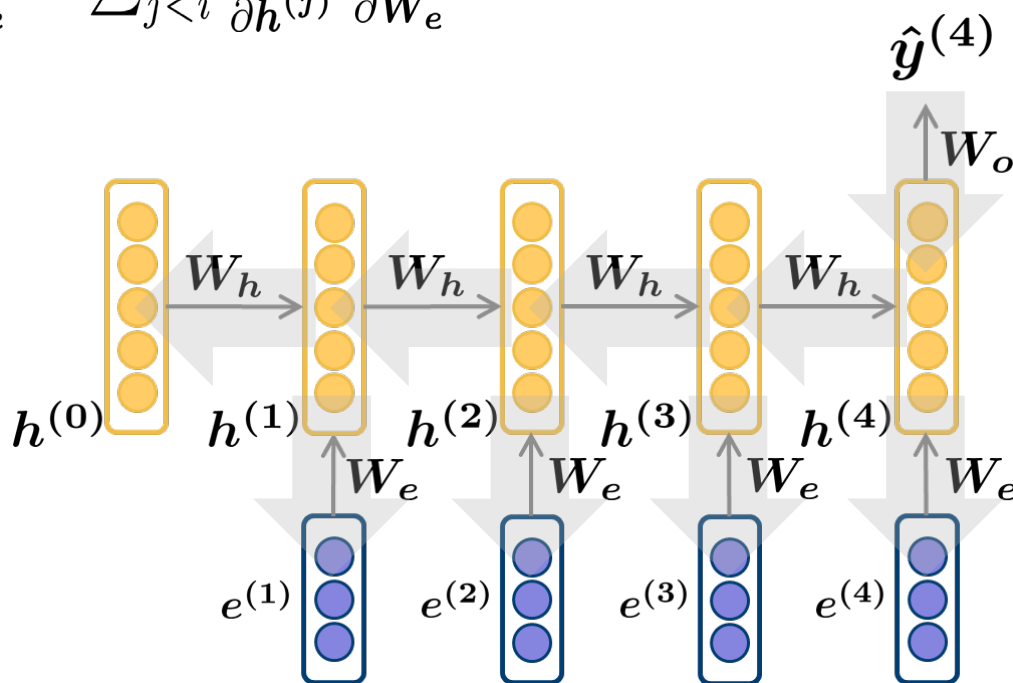
► 链式法则

循环神经网络的梯度计算

▶ 链式法则

$$\blacktriangleright \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{W}_h} = \sum_{j < i} \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(j)}} \frac{\partial \mathbf{h}^{(j)}}{\partial \mathbf{W}_h}$$

$$\blacktriangleright \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{W}_e} = \sum_{j < i} \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(j)}} \frac{\partial \mathbf{h}^{(j)}}{\partial \mathbf{W}_e}$$



► 链式法则

$$\text{► } \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{W}_h} = \sum_{j < i} \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(j)}} \frac{\partial \mathbf{h}^{(j)}}{\partial \mathbf{W}_h}$$

$$\text{► } \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{W}_e} = \sum_{j < i} \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(j)}} \frac{\partial \mathbf{h}^{(j)}}{\partial \mathbf{W}_e}$$

► 考虑公共项：

$$\frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(j)}} = \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(i)}} \prod_{j \leq t < i} \frac{\partial \mathbf{h}^{(t+1)}}{\partial \mathbf{h}^{(t)}}$$

循环神经网络的梯度计算

► 考虑: $\frac{\partial h^{(t+1)}}{\partial h^{(t)}}$

$$h^{(t+1)} = \sigma(W_h h^{(t)} + W_e e^{(t+1)} + b)$$

► 其中 σ 为激活函数

循环神经网络的梯度计算

► 考虑: $\frac{\partial h^{(t+1)}}{\partial h^{(t)}}$

$$h^{(t+1)} = \sigma(\tilde{h}^{(t+1)}) = \sigma(W_h h^{(t)} + W_e e^{(t+1)} + b)$$

► 其中 σ 为激活函数

► 考虑简化情况: 激活函数为线性 $\sigma(x) = x$

► 则

$$\begin{aligned}\frac{\partial h^{(t+1)}}{\partial h^{(t)}} &= \frac{\partial \sigma(\tilde{h}^{(t+1)})}{\partial \tilde{h}^{(t+1)}} \frac{\partial (W_h h^{(t)} + W_e e^{(t+1)} + b)}{\partial h^{(t)}} \\ &= 1 \times W_h\end{aligned}$$

► 于是

$$\begin{aligned}\frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(j)}} &= \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(i)}} \prod_{j \leq t < i} \frac{\partial \mathbf{h}^{(t+1)}}{\partial \mathbf{h}^{(t)}} \\ &= \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(i)}} \prod_{j \leq t < i} \mathbf{W}_h\end{aligned}$$

► 于是

$$\begin{aligned}\frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(j)}} &= \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(i)}} \prod_{j \leq t < i} \frac{\partial \mathbf{h}^{(t+1)}}{\partial \mathbf{h}^{(t)}} \\ &= \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(i)}} \prod_{j \leq t < i} \mathbf{W}_h\end{aligned}$$

► 继续简化：考虑标量情况

$$\frac{\partial \mathcal{L}^{(i)}}{\partial h^{(j)}} = \frac{\partial \mathcal{L}^{(i)}}{\partial h^{(i)}} \prod_{j \leq t < i} W_h = \frac{\partial \mathcal{L}^{(i)}}{\partial h^{(i)}} W_h^{i-j}$$

$$\frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(j)}} = \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(i)}} \prod_{j \leq t < i} W_h = \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(i)}} W_h^{i-j}$$

- ▶ 当 $W_h \neq 1$ 时，会出现什么情况？

$$\frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(j)}} = \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(i)}} \prod_{j \leq t < i} W_h = \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{h}^{(i)}} W_h^{i-j}$$

- ▶ 当 $W_h \neq 1$ 时，会出现什么情况？
- ▶ 类似的，将标量还原回向量、并引入激活函数，仍会存在这些问题。

- ▶ 梯度消失问题
 - ▶ 并不能有效的建立远距依赖关系
 - ▶ 例如：This kind of books is/are great.

循环神经网络存在的问题

- ▶ 梯度消失问题
 - ▶ 并不能有效的建立远距依赖关系
 - ▶ 例如：This kind of books is/are great.
- ▶ 梯度爆炸问题
 - ▶ 与神经网络中学习率过大时的情形类似

循环神经网络存在的问题

- ▶ 梯度消失问题

- ▶ 并不能有效的建立远距依赖关系
- ▶ 例如：This kind of books is/are great.

- ▶ 梯度爆炸问题

- ▶ 与神经网络中学习率过大时的情形类似
- ▶ 解决方法：clipping
 - ▶ 如果梯度的绝对值大于阈值，则将其绝对值设置为阈值，符号不变

- ▶ 为什么会出现梯度消失问题？

- ▶ 为什么会出现梯度消失问题?
 - ▶ 时间维度上的深度神经网络
 - ▶ RNN对于历史处理的共享假设加剧了这个现象

解决梯度消失问题

- ▶ 为什么会出现梯度消失问题？
 - ▶ 时间维度上的深度神经网络
 - ▶ RNN对于历史处理的共享假设加剧了这个现象
- ▶ 解决思路：
 - ▶ 向下一时刻传递信息时，考虑当前时刻的上下文情况

- ▶ 语言模型基本概念
- ▶ 统计语言模型回顾
- ▶ 神经网络语言模型
 - ▶ 前馈神经网络
 - ▶ 循环神经网络
- ▶ 长短时记忆网络LSTM及GRU
- ▶ 不同语言模型效果对比
- ▶ RNN/LSTM/GRU的应用

长短时记忆神经网络 (LSTM)

核心思想：

- ▶ 将长期记忆和短期记忆分离
 - ▶ 长期记忆用来存储和传递整个表达过程的状态，每一时刻根据上下文情况更新，且不能被外界直接观察
 - ▶ 短期记忆是根据当前上下文情况，从长期记忆中获取的，用以指导当前时刻的输出以及下一时刻长期记忆变化的部分，相当于当前时刻被“激活”的长期记忆

长短时记忆神经网络 (LSTM)

核心思想：

- ▶ 将长期记忆和短期记忆分离
 - ▶ 长期记忆：cell state $\mathbf{c}^{(t)} \in \mathbb{R}^n$
 - ▶ 短期记忆：hidden state $\mathbf{h}^{(t)} \in \mathbb{R}^n$

长短时记忆神经网络 (LSTM)

核心思想：

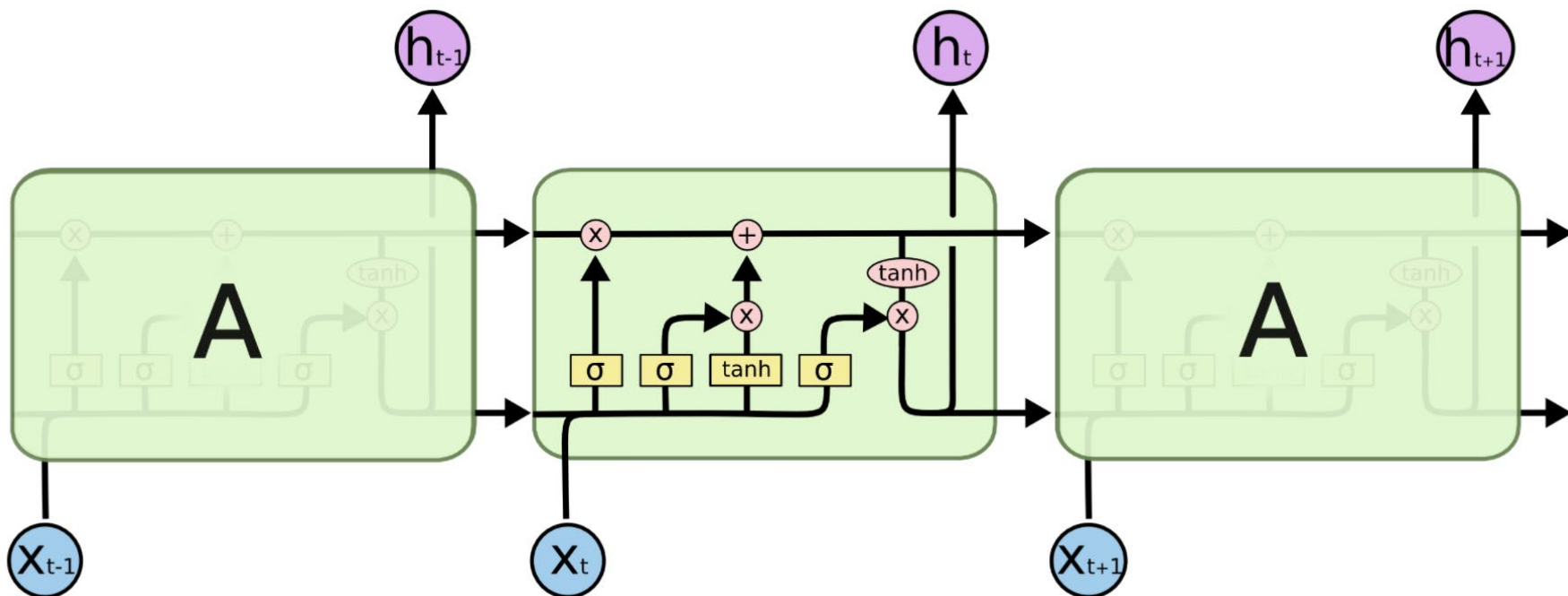
- ▶ 将长期记忆和短期记忆分离
 - ▶ 长期记忆：单元状态 $c^{(t)} \in \mathbb{R}^n$
 - ▶ 短期记忆：隐藏状态 $h^{(t)} \in \mathbb{R}^n$
- ▶ 使用门 (gate) 控制记忆的消除、传递与读写
 - ▶ 门是一个与记忆向量相同维度的向量
 - ▶ 每个位置是一个在0（关闭）和1（开启）间的实数
 - ▶ 在使用时，将门与其作用的向量做逐元素的乘法
 - ▶ 门是在每个时刻根据当前上下文动态生成的

- ▶ 在时间点 t ，输入的词向量为 $\mathbf{x}^{(t)}$ ，计算其隐藏状态 $\mathbf{h}^{(t)}$ 和单元状态 $\mathbf{c}^{(t)}$ ：
 - ▶ 遗忘门 $\mathbf{f}^{(t)}$ ：控制原单元状态中内容的保留与遗忘
 - ▶ 输入门 $\mathbf{i}^{(t)}$ ：控制当前时刻什么内容被写入单元状态
 - ▶ 输出门 $\mathbf{o}^{(t)}$ ：控制当前时刻什么内容从单元状态中被输出至隐藏状态

- ▶ 在时间点 t ，输入的词向量为 $\mathbf{x}^{(t)}$ ，计算其隐藏状态 $\mathbf{h}^{(t)}$ 和单元状态 $\mathbf{c}^{(t)}$ ：
 - ▶ 遗忘门： $\mathbf{f}^{(t)} = \sigma(\mathbf{W}_f[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_f)$
 - ▶ 输入门： $\mathbf{i}^{(t)} = \sigma(\mathbf{W}_i[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_i)$
 - ▶ 输出门： $\mathbf{o}^{(t)} = \sigma(\mathbf{W}_o[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_o)$

- ▶ 在时间点 t ，输入的词向量为 $\mathbf{x}^{(t)}$ ，计算其隐藏状态 $\mathbf{h}^{(t)}$ 和单元状态 $\mathbf{c}^{(t)}$ ：
 - ▶ 遗忘门： $\mathbf{f}^{(t)} = \sigma(\mathbf{W}_f[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_f)$
 - ▶ 输入门： $\mathbf{i}^{(t)} = \sigma(\mathbf{W}_i[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_i)$
 - ▶ 输出门： $\mathbf{o}^{(t)} = \sigma(\mathbf{W}_o[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_o)$
 - ▶ 候选单元状态： $\tilde{\mathbf{c}}^{(t)} = \tanh(\mathbf{W}_c[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_c)$
 - ▶ 单元状态： $\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \circ \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \circ \tilde{\mathbf{c}}^{(t)}$
 - ▶ 隐藏状态： $\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \circ \tanh(\mathbf{c}^{(t)})$

LSTM的流程图示



$$\begin{aligned}f^{(t)} &= \sigma(\mathbf{W}_f[h^{(t-1)}; x^{(t)}] + b_f) \\i^{(t)} &= \sigma(\mathbf{W}_i[h^{(t-1)}; x^{(t)}] + b_i) \\o^{(t)} &= \sigma(\mathbf{W}_o[h^{(t-1)}; x^{(t)}] + b_o) \\\tilde{c}^{(t)} &= \tanh(\mathbf{W}_c[h^{(t-1)}; x^{(t)}] + b_c) \\c^{(t)} &= f^{(t)} \circ c^{(t-1)} + i^{(t)} \circ \tilde{c}^{(t)} \\h^{(t)} &= o^{(t)} \circ \tanh(c^{(t)})\end{aligned}$$

- ▶ 思考：LSTM真的解决了梯度消失问题了吗？

- ▶ 思考：LSTM真的解决了梯度消失问题了吗？
- ▶ 确切的讲，LSTM让网络能够更容易的保留远距的信息，从而极大的缓解了梯度消失的问题
 - ▶ 朴素RNN需要 W_h 为单位矩阵以完全保留历史信息
 - ▶ LSTM只需要遗忘门为全1便可完全保留历史信息

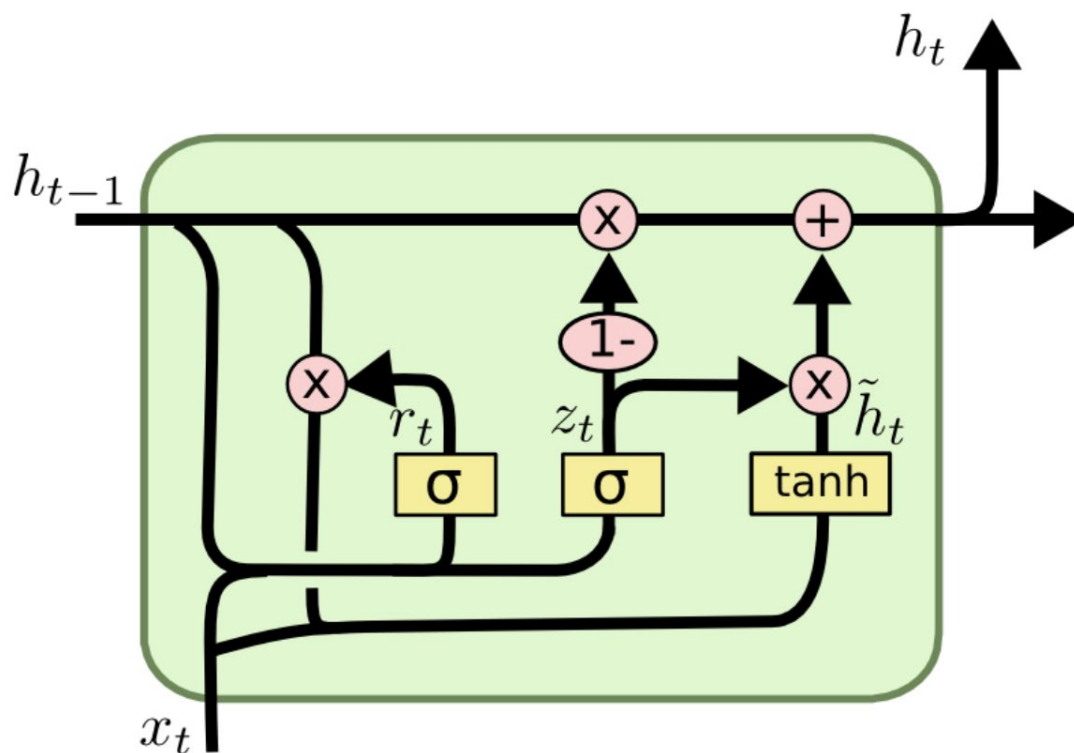
- ▶ 思考：LSTM真的解决了梯度消失问题了吗？
- ▶ 确切的讲，LSTM让网络能够更容易的保留远距的信息，从而极大的缓解了梯度消失的问题
 - ▶ 朴素RNN需要 W_h 为单位矩阵以完全保留历史信息
 - ▶ LSTM只需要遗忘门为全1便可完全保留历史信息
- ▶ 类似的：在DNN中的梯度消失问题与跳连
 - ▶ 学习单位矩阵变为学习全零矩阵

- ▶ 在时间点 t ，输入的词向量为 $\mathbf{x}^{(t)}$ ，计算其隐藏状态 $\mathbf{h}^{(t)}$ （无单元状态）
 - ▶ 更新门 $\mathbf{z}^{(t)}$ ：控制原隐藏状态中内容的更新与保留
 - ▶ 重置门 $\mathbf{r}^{(t)}$ ：控制原隐藏状态中什么部分被用来和当前时刻的输入一起计算新的隐藏状态内容

- ▶ 在时间点 t ，输入的词向量为 $\mathbf{x}^{(t)}$ ，计算其隐藏状态 $\mathbf{h}^{(t)}$ （无单元状态）
 - ▶ 更新门： $z^{(t)} = \sigma(\mathbf{W}_z[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_z)$
 - ▶ 重置门： $r^{(t)} = \sigma(\mathbf{W}_r[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_r)$

- ▶ 在时间点 t , 输入的词向量为 $\mathbf{x}^{(t)}$, 计算其隐藏状态 $\mathbf{h}^{(t)}$ (无单元状态)
 - ▶ 更新门: $\mathbf{z}^{(t)} = \sigma(\mathbf{W}_z[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_z)$
 - ▶ 重置门: $\mathbf{r}^{(t)} = \sigma(\mathbf{W}_r[\mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_r)$
 - ▶ 候选状态: $\tilde{\mathbf{h}}^{(t)} = \tanh(\mathbf{W}_h[\mathbf{r}^{(t)} \circ \mathbf{h}^{(t-1)}; \mathbf{x}^{(t)}] + \mathbf{b}_h)$
 - ▶ 隐藏状态: $\mathbf{h}^{(t)} = (1 - \mathbf{z}^{(t)}) \circ \mathbf{h}^{(t-1)} + \mathbf{z}^{(t)} \circ \tilde{\mathbf{h}}^{(t)}$

简化LSTM: GRU



$$z^{(t)} = \sigma(W_z[h^{(t-1)}; x^{(t)}] + b_z)$$

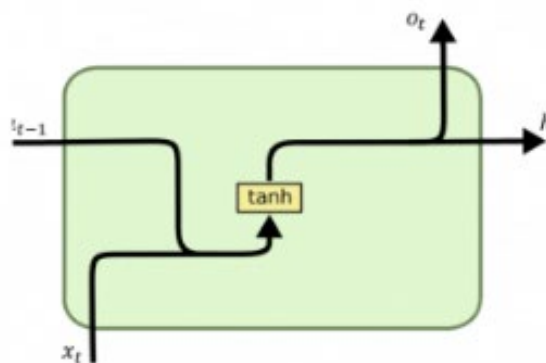
$$r^{(t)} = \sigma(W_r[h^{(t-1)}; x^{(t)}] + b_r)$$

$$\tilde{h}^{(t)} = \tanh(W_h[r^{(t)} \circ h^{(t-1)}; x^{(t)}] + b_h)$$

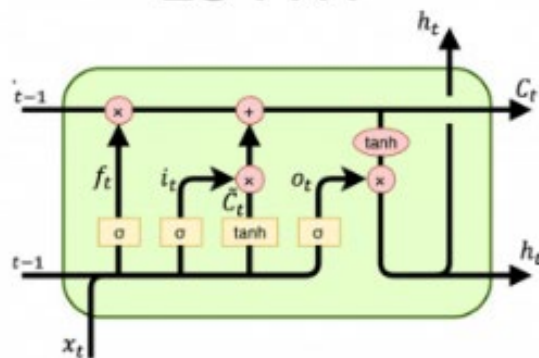
$$h^{(t)} = (1 - z^{(t)}) \circ h^{(t-1)} + z^{(t)} \circ \tilde{h}^{(t)}$$

RNN, LSTM和GRU的对比

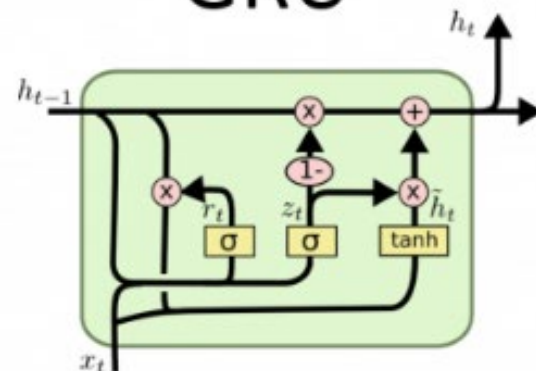
RNN



LSTM



GRU



<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

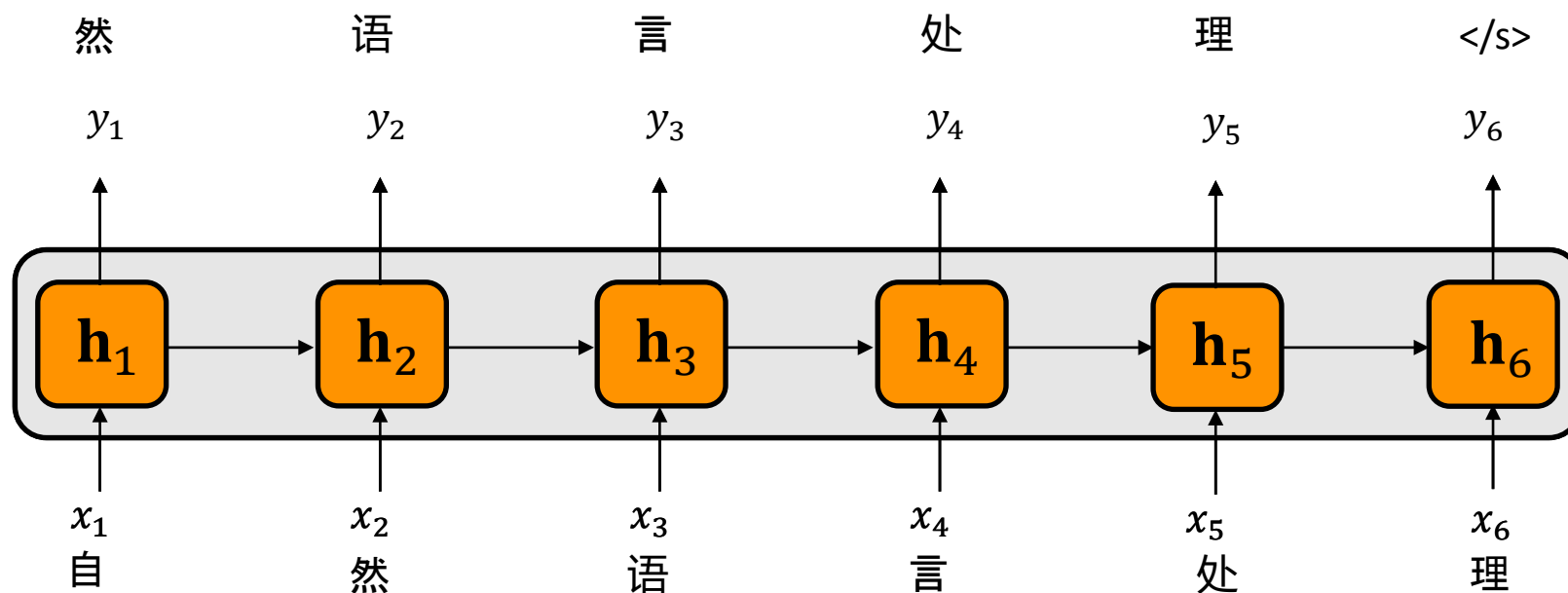
- ▶ 语言模型基本概念
- ▶ 统计语言模型回顾
- ▶ 神经网络语言模型
 - ▶ 前馈神经网络
 - ▶ 循环神经网络
- ▶ 长短时记忆网络LSTM及GRU
- ▶ 不同语言模型效果对比
- ▶ RNN/LSTM/GRU的应用

神经网络语言模型效果对比

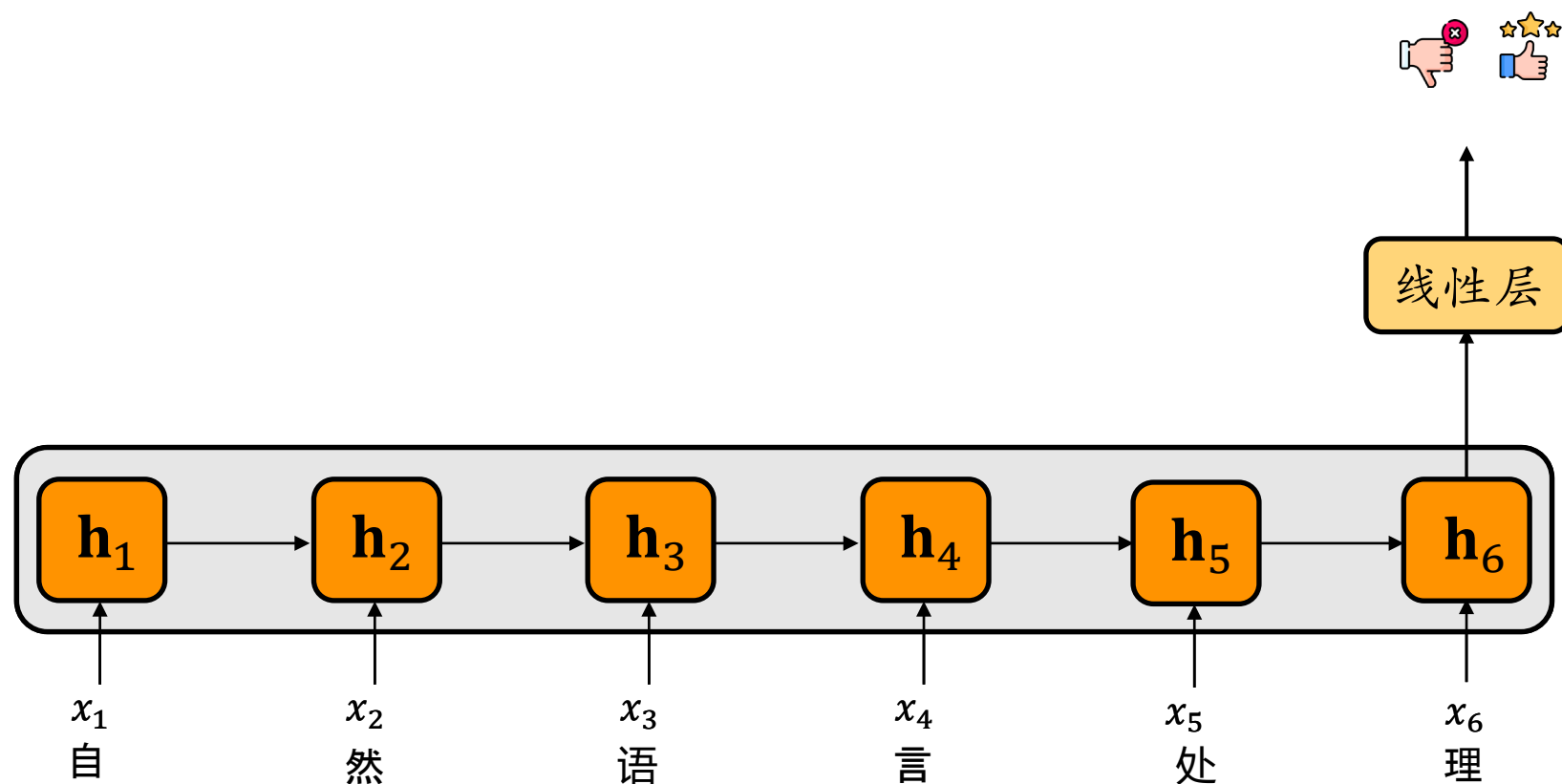
模型	PPL
3-gram	206.42
4-gram	160.68
5-gram	172.63
RNN	147.57
LSTM	78.63
GRU	78.82

- ▶ 语言模型基本概念
- ▶ 统计语言模型回顾
- ▶ 神经网络语言模型
 - ▶ 前馈神经网络
 - ▶ 循环神经网络
- ▶ 长短时记忆网络LSTM及GRU
- ▶ 不同语言模型效果对比
- ▶ RNN/LSTM/GRU的应用

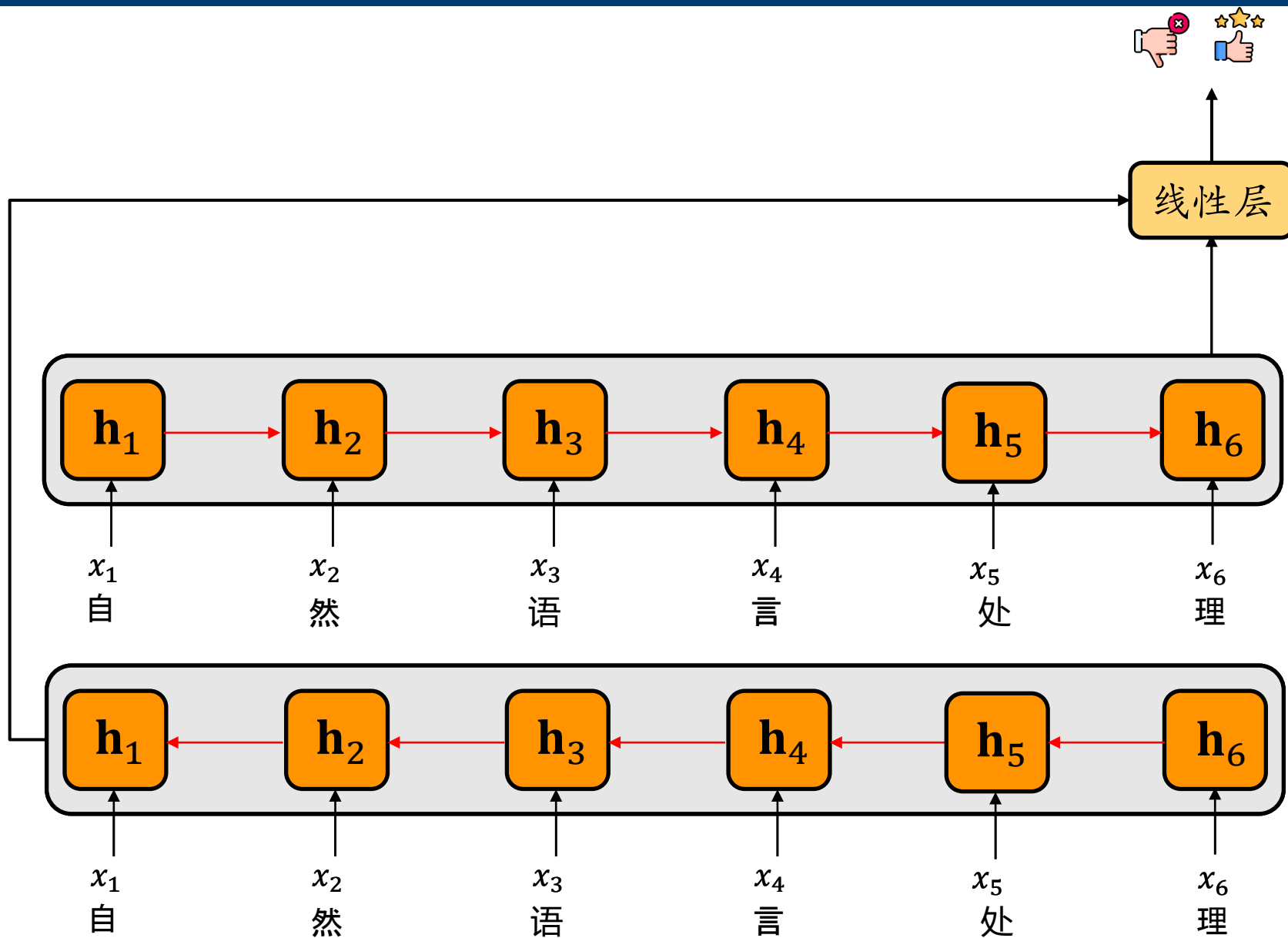
RNN/LSTM/GRU的应用：语言模型



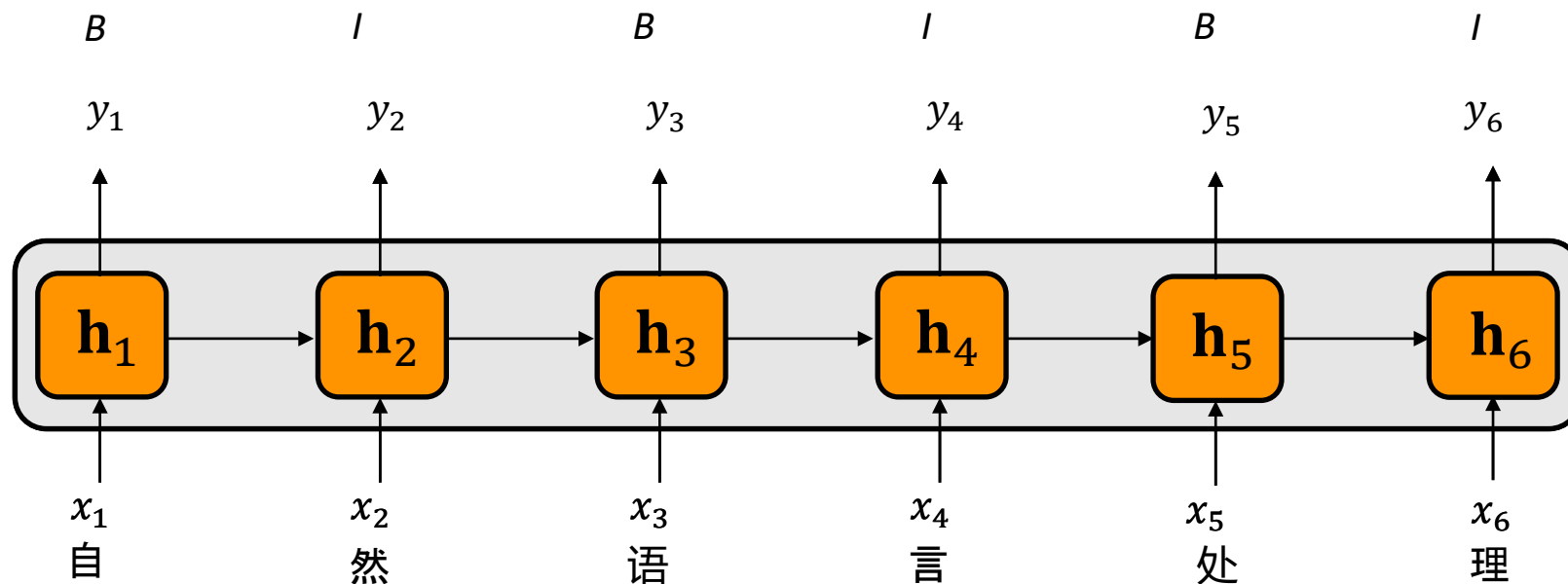
RNN/LSTM/GRU的应用：文本分类



RNN/LSTM/GRU的应用：文本分类



RNN/LSTM/GRU的应用：序列标注



RNN/LSTM/GRU的应用：序列到序列生成

